Modul 2

Pewarisan (Inheritance)

Praktikum Pemrograman Berorientasi Objek 2026

Universitas Atma Jaya Yogyakarta

Fakultas Teknologi Industri

Program Studi Informatika

Pewarisan (Inheritance)

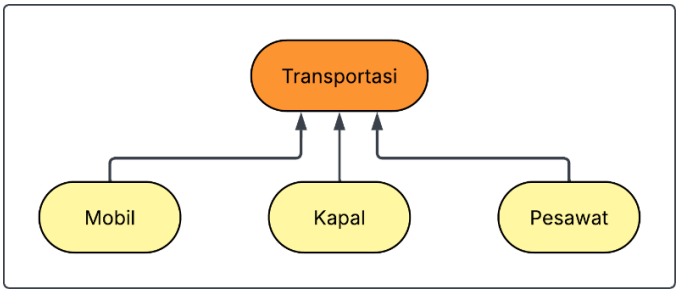
Pewarisan atau inheritance merupakan salah satu dari lima pilar OOP **(abstraksi, enkapsulasi, pewarisan, polimorfisme, dan relasi)**. Pewarisan dalam OOP menjadi salah satu konsep yang penting. Mengapa demikian? Karena kita bisa meminimalisir penulisan kode yang berulang-ulang baik pada fungsi maupun variabel. Hal ini dapat terjadi karena pewarisan merupakan konsep OOP dimana sebuah kelas atau objek dapat mewarisi sifat dan perilaku kelas atau objek lain. Dengan menggunakan pewarisan, sebuah kelas mempunyai kesempatan untuk menggunakan atribut dan method dari kelas lain.

Istilah dalam pewarisan:

- **SuperClass** : Sebuah kelas yang akan mewariskan fitur-fitur berupa atribut maupun method. Istilah lain untuk superclass yaitu *base class* dan *parent class* (termasuk terjemahan Bahasa Indonesianya).
- **SubClass** : Kelas yang akan mewarisi fitur-fitur dari superclass. Namun, perlu diingat bahwa subclass juga harus memiliki fiturnya sendiri selain yang diwarisinya. Istilah lain untuk subclass yaitu *child class* dan *derived class* (termasuk terjemahan bahasa Indonesianya).
- **Reusability** : Kemampuan untuk menggunakan kembali kemampuan yang sudah ada. Pewarisan merupakan salah satu perwujudan reusability.

Analogi Pewarisan Dalam Keseharian

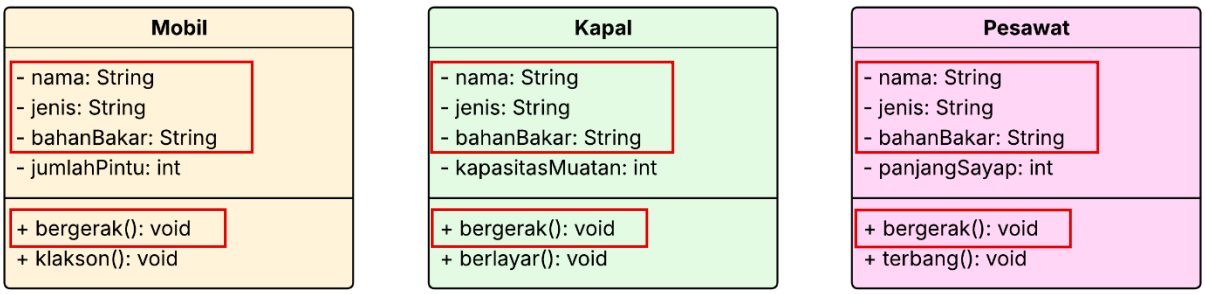
Apa yang kalian bayangkan ketika mendengar kata Mobil, Kapal, dan Pesawat? Pastinya kita akan dapat menemukan persamaan dan perbedaan diantara ketiga objek tadi. Mobil, Kapal, dan Pesawat sama-sama mempunyai kemiripan sebagai transportasi dan juga mempunyai beberapa karakteristik yang sama, seperti sama-sama memiliki nama, jenis, bahan bakar, dan lain-lain. Namun, ketiga objek tadi juga tentu memiliki perbedaan. Mobil hanya digunakan di darat, Pesawat digunakan di udara, dan Kapal digunakan di laut. Berdasarkan informasi yang kita miliki, kita dapat mengelompokkan ketiga objek tadi dengan struktur hierarkis seperti contoh pada Gambar 1.



Gambar 1. Struktur hierarki kelas Transportasi

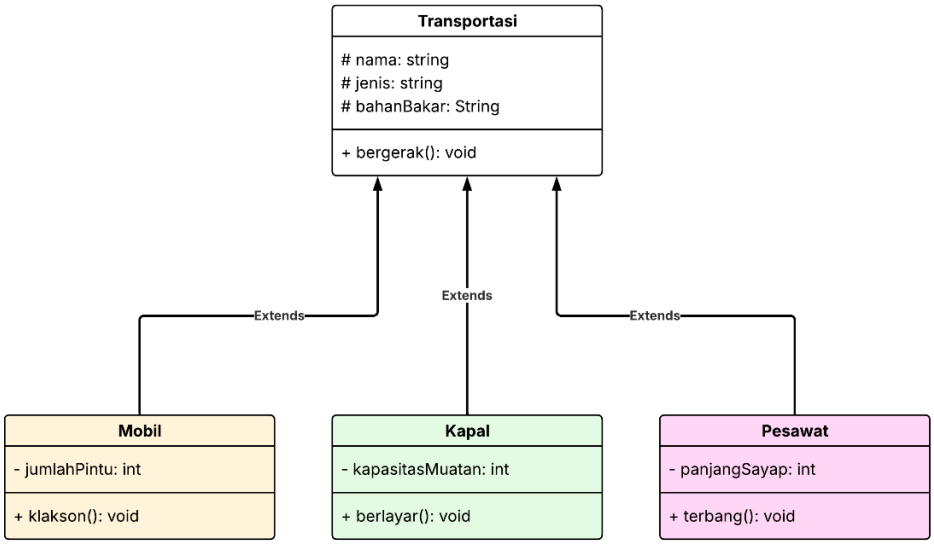
Baik Mobil, Kapal, maupun Pesawat, sama-sama merupakan transportasi. Maka ketiga objek tersebut adalah turunan (inheritance) dari transportasi yang mewarisi karakteristik transportasi. Sebagai turunan transportasi, ketiga objek tersebut juga tetap memiliki karakteristiknya masing-masing.

Manfaat Pewarisan



Gambar 2. Diagram kelas Mobil, Kapal, dan Pesawat

Perhatikan Diagram kelas Mobil, Kapal, dan Pesawat yang ditunjukkan pada Gambar 2. Pada masing-masing kelas terdapat atribut dan method yang ditulis berulang-ulang. Hal ini menyebabkan duplikasi kode dan akan menyulitkan pemeliharaan kode kedepannya. Contohnya jika kita ingin mengubah atribut pada sebuah kelas, maka kita harus mengubah atribut pada kelas lainnya juga. Lalu bagaimana solusinya? Pada kasus inilah kita membutuhkan pewarisan. Perhatikan Diagram kelas pada Gambar 3.



Gambar 3. Diagram kelas Transportasi, Mobil, Kapal, dan Pesawat

Pada Gambar 3, tampak bahwa kita menggeneralisir kesamaan atribut dan method pada kelas Mobil, Kapal, dan Pesawat menjadi sebuah kelas baru dengan bentuk yang lebih umum yaitu kelas transportasi. Kini kelas transportasi menjadi superclass, sedangkan kelas Mobil, Kapal, dan Pesawat menjadi subclass dari kelas transportasi yang mewariskan atribut atau methodnya namun juga tetap memiliki atribut atau methodnya sendiri. Proses yang kita lakukan ketika kita mengelompokkan kesamaan pada ketiga kelas dan kemudian membentuknya menjadi sebuah kelas baru yang lebih umum sebagai superclass ini disebut **generalisasi**. Sedangkan jika kita membuat subclass baru dari sebuah superclass yang sudah ada disebut **spesialisasi**.

Dari studi kasus sebelumnya, kini kita dapat memperoleh beberapa manfaat dari pewarisan, yaitu:

- Menerapkan reusability pada kode. Dengan menggunakan kembali suatu fungsionalitas yang sudah ada sebelumnya, kita dapat menghindari duplikasi kode.
- Struktur program lebih terorganisir. Dengan menerapkan pewarisan, kita akan memperoleh struktur hierarkis yang meningkatkan keterbacaan kode serta struktur kode yang lebih modular.
- Mempermudah pemeliharaan kode. Jika kelas induk mengalami perubahan, maka semua kelas anak otomatis akan memperolehnya. Hal ini akan menguntungkan kita ketika kita ingin mengubah suatu kelas, kita tidak perlu mengubahnya satu per satu. Terakhir, kita juga akan lebih mudah dalam menemukan bug pada kode nantinya.

Hal Teknis Pada Pewarisan

Setelah kita memahami teori terkait pewarisan, sekarang kita akan mempelajari hal-hal yang lebih teknis. Mari kita bahas satu per satu.

Visibilitas Protected (#)

Dalam pewarisan, atribut dan method yang akan diwariskan oleh superclass adalah yang visibilitasnya **public(+)** dan **protected(#)** saja, sedangkan untuk yang visibilitasnya **private(-)** tidak diwariskan. Jika kalian perhatikan, pada superclass terdapat atribut yang access modifiernya (visibilitas) menggunakan protected (#). Dalam OOP, suatu member class yang memiliki access modifier **protected(#)** dapat diakses oleh **kelas itu sendiri** dan **kelas yang menjadi anaknya**. Konsep ini diterapkan juga pada bahasa lain yang mendukung OOP seperti C++, C#, PHP, dan lain-lain. Dalam OOP, kita akan tetap berpegang pada konsep tersebut untuk mendukung enkapsulasi.

Tambahan Tentang Protected(#) Pada Java

Java sendiri memiliki konsep yang sedikit unik terkait visibilitas protected(#). Dalam java, member dari suatu kelas yang memiliki visibilitas protected selain dapat diakses oleh kelas turunan baik yang ada di package yang sama maupun berbeda dari parent class, ternyata dapat diakses oleh semua kelas dalam package yang sama sekalipun kelas itu bukan turunan dari suatu kelas induk. Itulah mengapa kita bisa mengakses atribut protected meskipun bukan dari kelas turunan. Tapi kita tidak akan terlalu membahas hal tersebut, karena ini hanya menjadi pengetahuan tambahan saja. Kalian dapat mengeksplor sendiri atau mengunjungi dokumentasi resmi dari java pada Controlling Access to Members of a Class (The Java™ Tutorials > Learning the Java Language > Classes and Objects).

Relasi "Is - A"

Sekedar tambahan, hubungan antara kelas anak dengan kelas induk pada pewarisan merupakan relasi "is-a". Jika sebuah subclass mewarisi superclass, maka dapat dipastikan bahwa subclass "is-a" superclass. Contohnya Mobil "is-a" transportasi. Materi ini akan dibahas lebih lanjut pada modul relasi.

Keyword Extends dan super()

Untuk melakukan pewarisan dari kelas induk, biasanya kita menggunakan keyword extends saat membuat kelas. Sedangkan untuk memanggil konstruktor maupun method dari

kelas induk, kita menggunakan keyword super() pada kelas anak. Perhatikan contoh pada Gambar 4.

```java
public class Transportasi {
    protected String nama;
    protected String jenis;
    protected String bahanBakar;

    public Transportasi(String nama, String jenis, String bahanBakar) {
        this.nama = nama;
        this.jenis = jenis;
        this.bahanBakar = bahanBakar;
    }

    public void bergerak(){
        System.out.println(this.nama +" berjenis " +this.jenis +" sedang bergerak");
    }
}
```

Gambar 4. Kelas Transportasi

Pada kelas transportasi diatas, terlihat bahwa kita menggunakan visibilitas protected(#) pada atribut nama, jenis, dan bahanBakar. Ini akan memastikan bahwa atribut-atribut tersebut hanya dapat diakses oleh kelas itu sendiri dan kelas turunannya. Sementara itu, konstruktor dan method public tetap dapat diakses oleh kelas anak.

```java
public class Mobil extends Transportasi{
    // sisa kode belum ditulis
}
```

Gambar 5. Kelas Mobil

Sekarang kita memiliki kelas Mobil yang ditunjukkan pada Gambar 5. Kelas Mobil akan melakukan pewarisan dari kelas transportasi. Untuk melakukan pewarisan, subclass perlu menggunakan keyword **extends** diikuti nama **superclass**nya.

```java
public class Mobil extends Transportasi{
    private int jumlahPintu;

    public Mobil(int jumlahPintu, String nama, String jenis, String bahanBakar) {
        super(nama, jenis, bahanBakar);
        this.jumlahPintu = jumlahPintu;
    }
}
```

Gambar 6. Kelas Mobil dengan konstruktor

Perlu diketahui bahwa **konstruktor induk tidak diwariskan**, sehingga kelas anak perlu membuat konstruktor sendiri. Namun, kita tetap dapat memanggil konstruktor kelas induk

dengan keyword **super**. Sekarang perhatikan kelas Mobil pada Gambar 6, kita ingin membuat konstruktor kelas Mobil dengan tambahan atribut milik kelas Mobil.

Beberapa hal yang perlu diperhatikan dalam membuat konstruktor anak:

- **Perhatikan parameter konstruktor**, kita tetap perlu menuliskan parameter konstruktor induknya ditambah parameter untuk inisialisasi atribut kelas itu sendiri.
- **Urutan penulisan parameter**, biasanya penulisan parameter dimulai dari atribut konstruktor kelas induk, baru kemudian dilanjutkan dengan atribut kelas anak. Ini akan meningkatkan keterbacaan kode.
- **Untuk memanggil konstruktor kelas induk**, kita cukup menuliskan super(parameter1, parameter2, ...) di dalam konstruktor anak.

```java
public class Mobil extends Transportasi{
    private int jumlahPintu;

    public Mobil(int jumlahPintu, String nama, String jenis, String bahanBakar) {
        super(nama, jenis, bahanBakar);
        this.jumlahPintu = jumlahPintu;
    }

    public void klakson(){
        super.bergerak();
        System.out.println("dengan membunyikan klakson");
    }
}
```

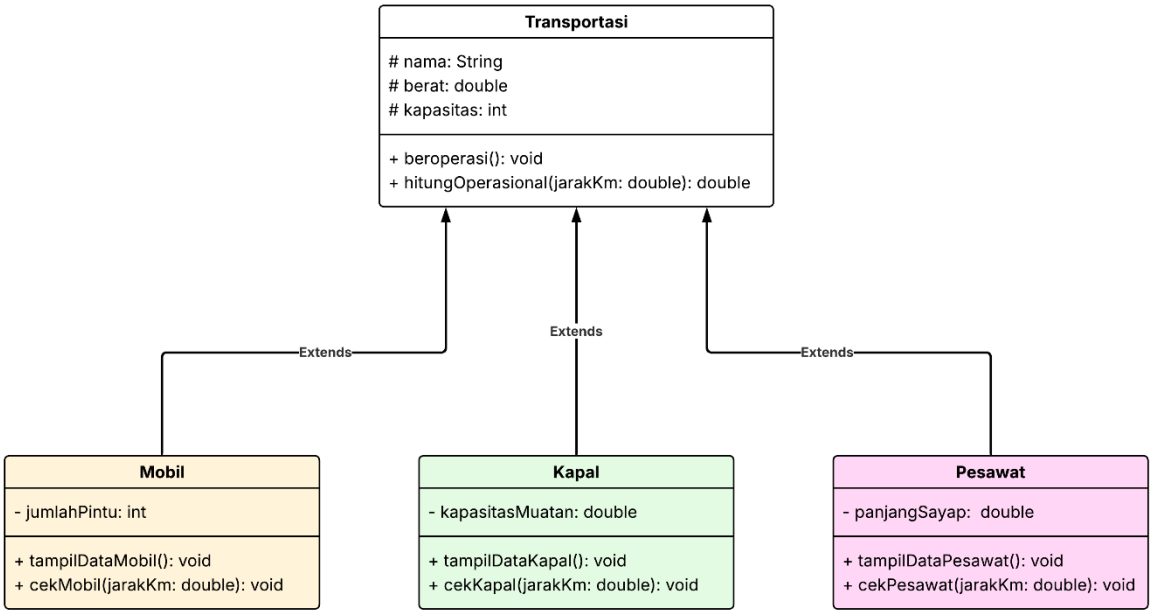
Gambar 7. Kelas Mobil dengan method warisan

Terakhir, kita juga dapat memanggil method kelas induk di kelas anak. Caranya cukup mudah yaitu dengan menuliskan super.namaMethod() seperti pada Gambar 7.

Nah, itu dia teori mengenai pewarisan yang sudah kita bedah sejauh ini. Agar lebih memahami penerapannya secara langsung, sekarang mari kita mengerjakan guided.

Guided Pewarisan

Pada modul kali ini, kita akan mengerjakan 2 guided, yaitu guided dengan menggunakan BlueJ dan Netbeans. Maka dari itu, mari perhatikan diagram class pewarisan dibawah ini.

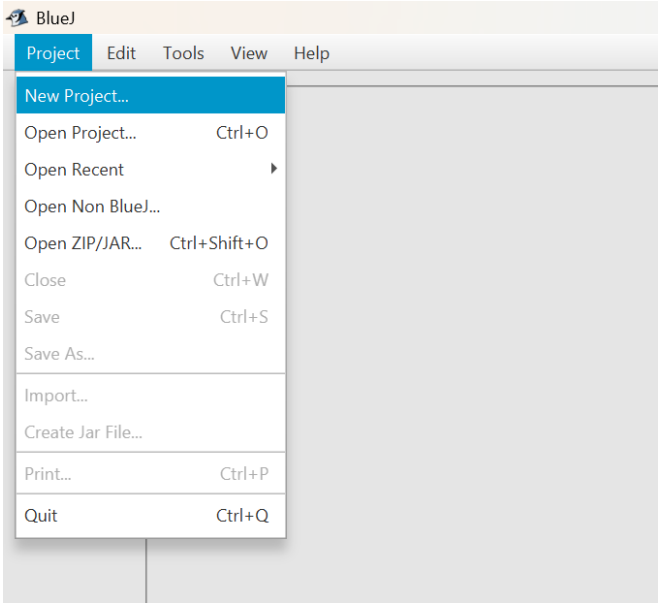


Seperti yang terlihat pada gambar, Transportasi akan menjadi kelas induk yang akan diwariskan kepada kelas anak yaitu Mobil, Kapal, dan Pesawat. Tujuan dari praktikum ini adalah untuk menerapkan konsep pewarisan kedalam bentuk program, sehingga praktikan memahami bagaimana sebuah kelas diwariskan, bagaimana kelas anak mewarisi kelas induk, dan apa yang dapat kita lakukan dalam menerapkan pewarisan.

Source code untuk BlueJ dan Netbeans sama, jadi nanti kita akan menggunakan BlueJ untuk memvisualisasikan bagaimana pewarisan bekerja, sedangkan untuk penjelasan kode akan dijelaskan pada guided Netbeans.

GUIDED BLUEJ

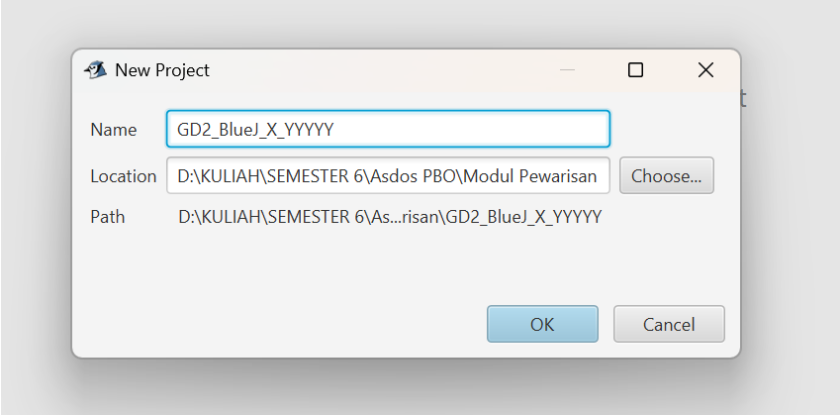
1. Buatlah project baru pada BlueJ dengan cara klik **Project > New Project**



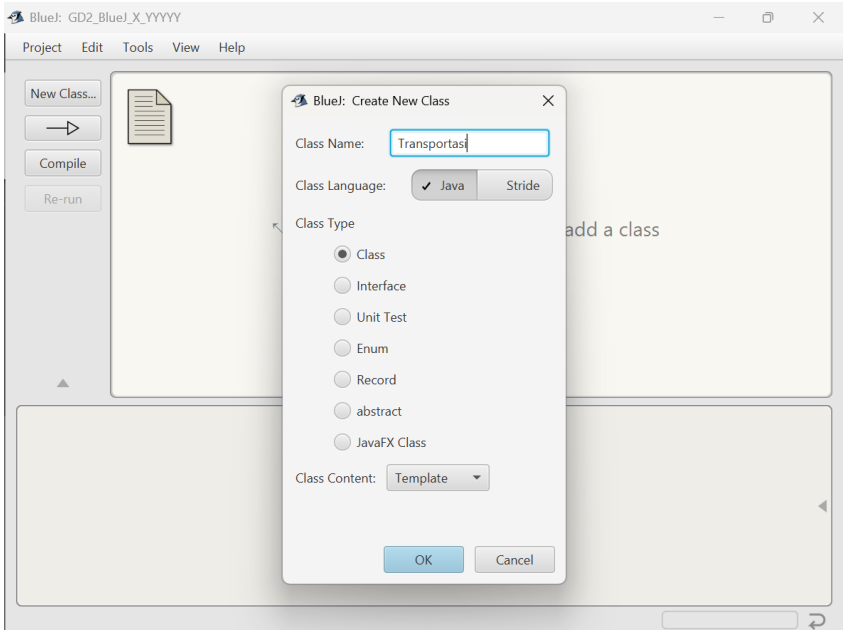
Kemudian, sesuaikan nama file GD2_BlueJ_X_YYYYY

X = Kelas

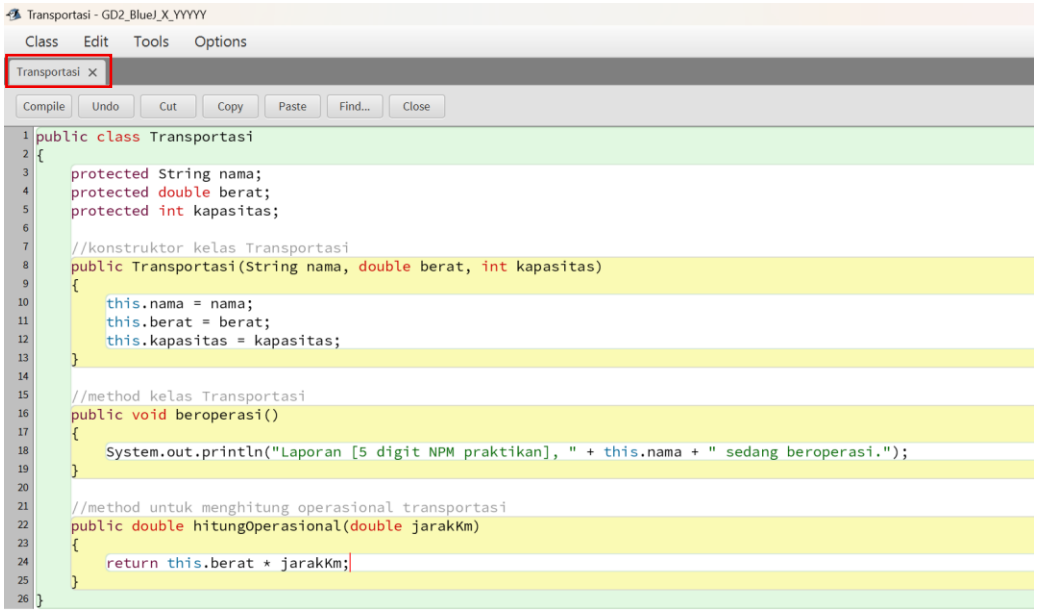
Y = 5 digit NPM praktikan



2. Buatlah kelas Transportasi sebagai kelas induk. Klik **New Class**, kemudian beri nama Transportasi



Setelah itu, klik 2 kali pada kelas transportasi, maka nanti akan tampil source code untuk kelas transportasi. Isikan kode berikut pada halaman tersebut. Jika sudah menuliskan code, tutup jendela source code dengan klik close tab.




```java
public class Transportasi
{
    protected String nama;
    protected double berat;
    protected int kapasitas;

    //konstruktor kelas Transportasi
    public Transportasi(String nama, double berat, int kapasitas)
    {
        this.nama = nama;
        this.berat = berat;
        this.kapasitas = kapasitas;
    }

    //method kelas Transportasi
    public void beroperasi()
    {
        System.out.println("Laporan [5 digit NPM praktikan], " + this.nama + " sedang beroperasi.");
    }

    //method untuk menghitung operasional transportasi
    public double hitungOperasional(double jarakKm)
    {
        return this.berat * jarakKm;
    }
}
```

3. Setelah membuat kelas Transportasi, buat lagi tiga buah kelas bernama Mobil, Kapal, dan Pesawat lalu tuliskan kode berikut pada masing masing souce code kelas.

Kelas Mobil

Mobil - GD2_BlueJ_X_YYYYY

Class Edit Tools Options

Mobil

Compile Undo Cut Copy Paste Find... Close Source Code

```java
public class Mobil extends Transportasi
{
    private int jumlahPintu;

    public Mobil(String nama, double berat, int kapasitas, int jumlahPintu)
    {
        super(nama, berat, kapasitas);
        this.jumlahPintu = jumlahPintu;
    }

    public void tampilDataMobil()
    {
        System.out.println("Nama            : " + nama);
        System.out.println("Berat           : " + berat + " kg");
        System.out.println("Kapasitas       : " + kapasitas + " orang");
        System.out.println("Jumlah Pintu    : " + jumlahPintu + " pintu");
    }

    public void cekMobil(double jarakKm)
    {
        double bobot = hitungOperasional(jarakKm) / 1000;
        System.out.println(this.nama + " memiliki bobot mobil: " + bobot);

        if(bobot < 5000)
        {
            System.out.println("Kategori: Mobil Ringan");
        }
        else
        {
            System.out.println("Kategori: Mobil Berat");
        }
    }
}
```

Kelas Kapal

Kapal - GD2_BlueJ_X_YYYYY

Class Edit Tools Options

Kapal

Compile Undo Cut Copy Paste Find... Close Source Code

```java
public class Kapal extends Transportasi
{
    private double kapasitasMuatan;

    public Kapal(String nama, double berat, int kapasitas, double kapasitasMuatan)
    {
        super(nama, berat, kapasitas);
        this.kapasitasMuatan = kapasitasMuatan;
    }

    public void tampilDataKapal()
    {
        System.out.println("Nama            : " + nama);
        System.out.println("Berat           : " + berat + " kg");
        System.out.println("Kapasitas       : " + kapasitas + " orang");
        System.out.println("Kapasitas Muatan: " + kapasitasMuatan + " kg");
    }

    public void cekKapal(double jarakKm)
    {
        double operasional = hitungOperasional(jarakKm) + (kapasitasMuatan * 500);
        System.out.println(this.nama + " memiliki operasional kapal: " + operasional);

        if(operasional < 500000)
        {
            System.out.println("Kategori: Stabil");
        }
        else
        {
            System.out.println("Kategori: Berat");
        }

    }
}
```

Kelas Pesawat

Pesawat - GD2_BlueJ_X_YYYYY

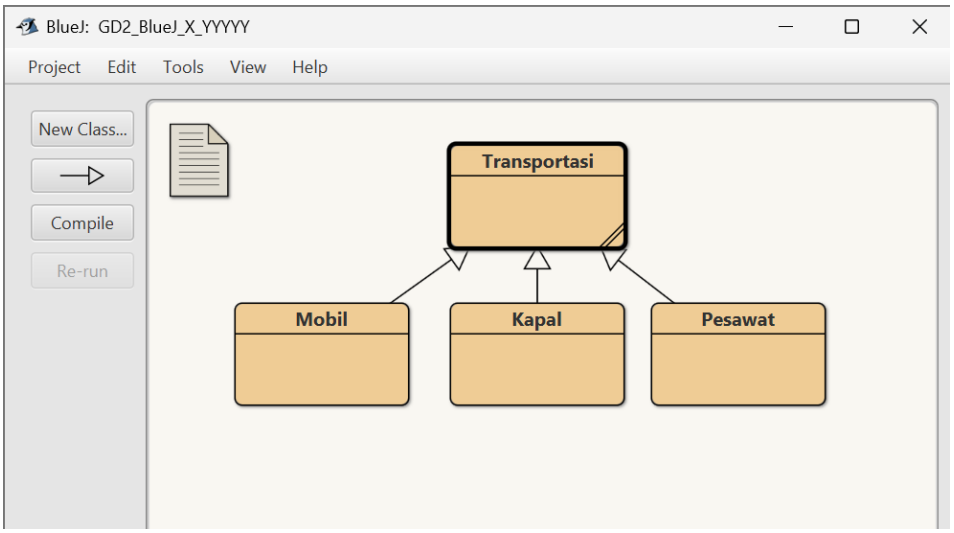
Class Edit Tools Options

Pesawat

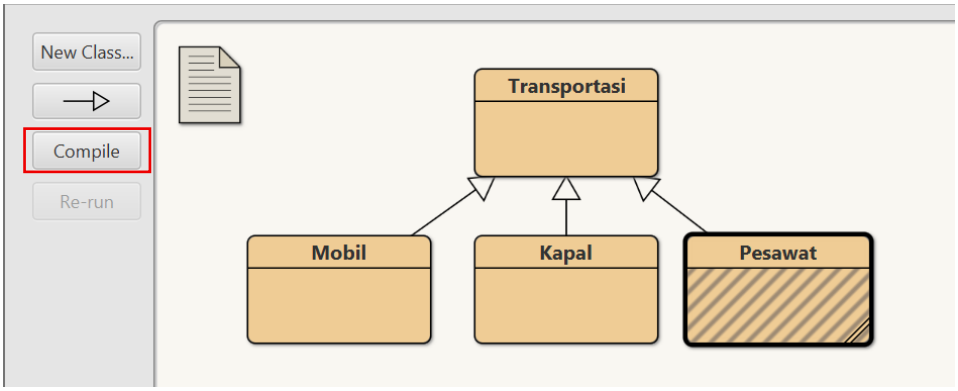
Compile Undo Cut Copy Paste Find... Close Source Code

```java
public class Pesawat extends Transportasi
{
    private double panjangSayap;

    public Pesawat(String nama, double berat, int kapasitas, double panjangSayap)
    {
        super(nama, berat, kapasitas);
        this.panjangSayap = panjangSayap;
    }

    public void tampilDataPesawat()
    {
        System.out.println("Nama            : " + nama);
        System.out.println("Berat           : " + berat + " kg");
        System.out.println("Kapasitas       : " + kapasitas + " orang");
        System.out.println("Panjang Sayap   : " + panjangSayap + " m");
    }

    public void cekPesawat(double jarakKm)
    {
        double operasional = hitungOperasional(jarakKm) + (kapasitas * 500) + (panjangSayap * 200);
        System.out.println(this.nama + " memiliki operasional pesawat: " + operasional);

        if(operasional <= 1500000)
        {
            System.out.println("Kategori: Standar");
        }
        else
        {
            System.out.println("Kategori: Tinggi");
        }

    }
}
```

4. Setelah selesai menyalin kode diatas, perhatikan struktur kelas kalian.

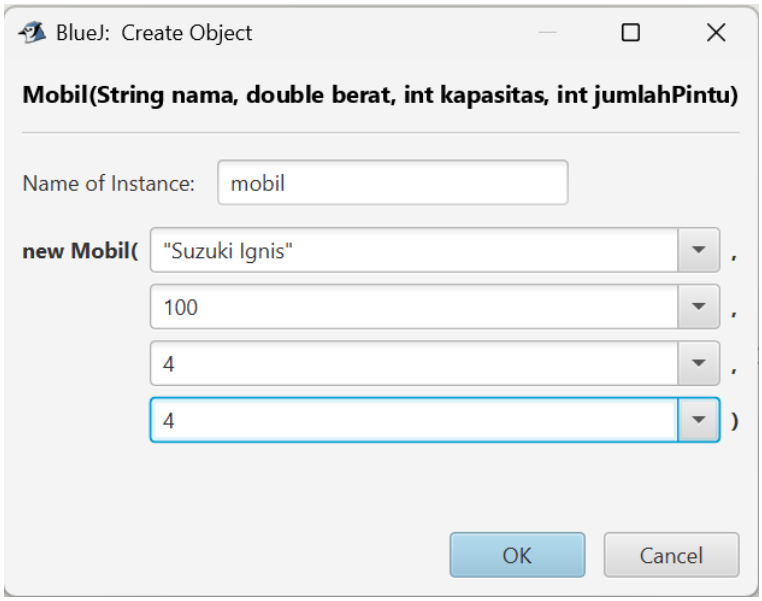


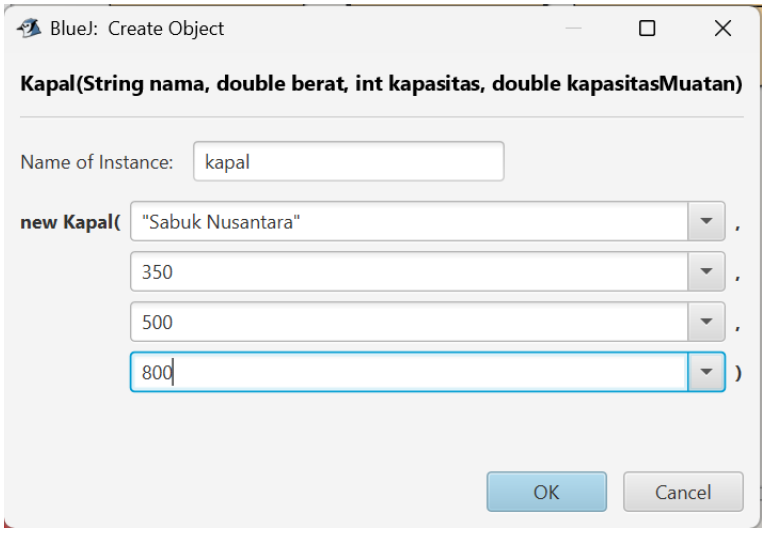
Ketika strukturnya sudah sama dengan gambar, maka kelas yang kalian buat sudah benar.

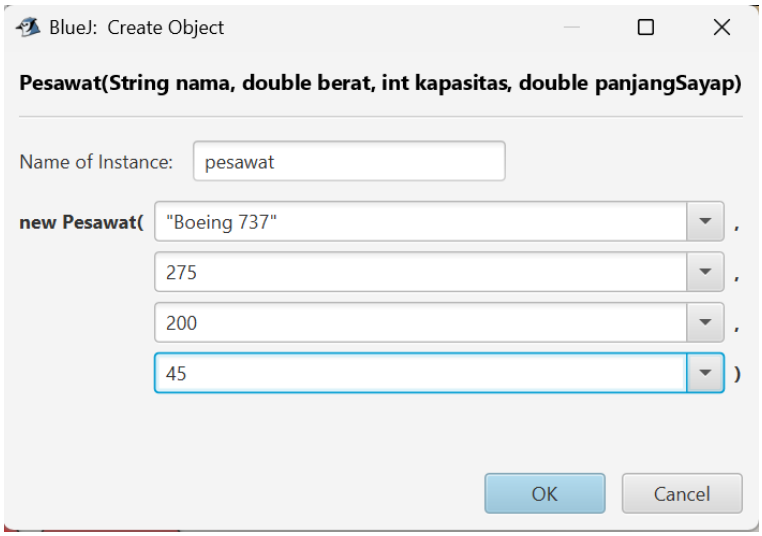


Sedikit catatan untuk BlueJ, jika kalian mendapati kelas yang bergaris garis seperti pada gambar, artinya kalian belum mengcompile kelas tersebut. Untuk mengatasinya, cukup tekan tombol compile.

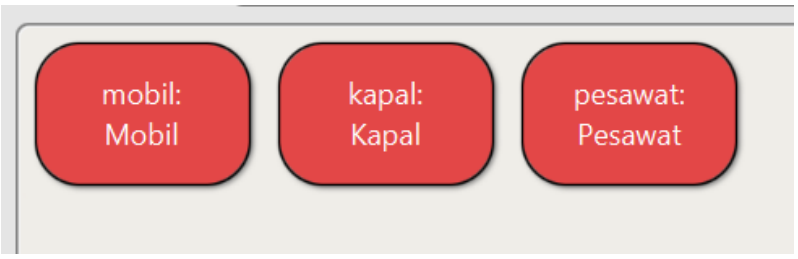
5. Kemudian, buatlah objek untuk masing masing kelas. Klik kanan pada kelas, dan pilih keyword new. Buat 1 objek Mobil, 1 objek Kapal, dan 1 objek Pesawat.



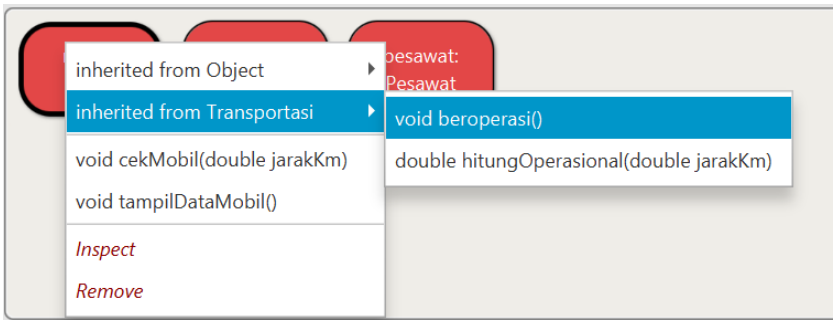




Ketika sudah, nantinya objek yang kita buat akan tampak seperti ini.



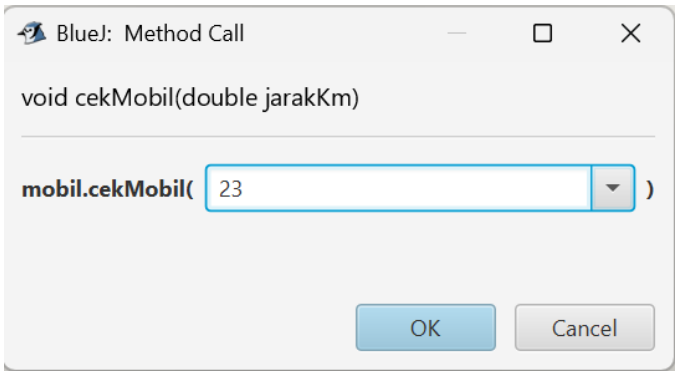
6. Uji coba method beroperasi() pada semua objek.

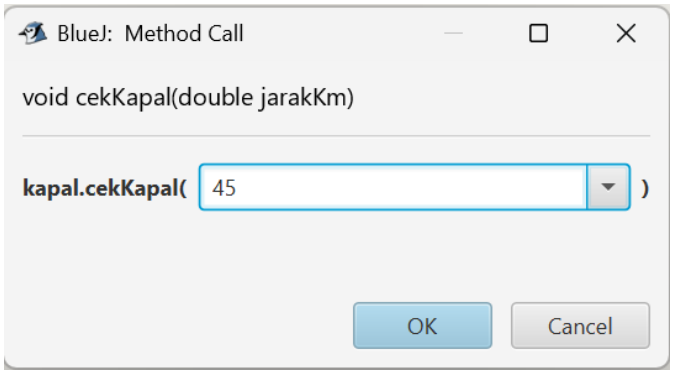


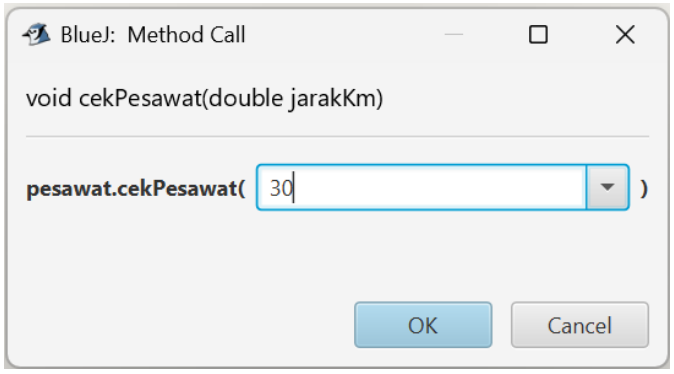
Output akan tampil seperti ini

```
Laporan [5 digit NPM praktikan], Suzuki Ignis sedang beroperasi.
Laporan [5 digit NPM praktikan], Sabuk Nusantara sedang beroperasi.
Laporan [5 digit NPM praktikan], Boeing 737 sedang beroperasi.
```

7. Uji coba method hitungOperasional() pada semua objek. Masukkan inputan angka:







8. Gunakan method kelas anak untuk menampilkan output berikut

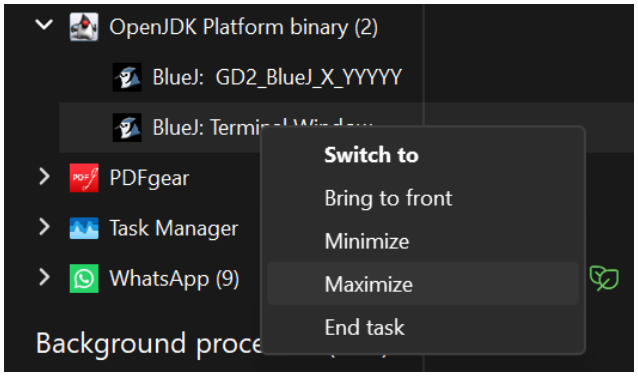
```
Suzuki Ignis memiliki bobot mobil: 2.3
Kategori: Mobil Ringan
Sabuk Nusantara memiliki operasional kapal: 415750.0
Kategori: Stabil
Boeing 737 memiliki operasional pesawat: 117250.0
Kategori: Standar
```

9. Method tampilData() pada setiap objek

```
Nama            : Suzuki Ignis
Berat           : 100.0 kg
Kapasitas       : 4 orang
Jumlah Pintu    : 4 pintu
Nama            : Sabuk Nusantara
Berat           : 350.0 kg
Kapasitas       : 500 orang
Kapasitas Muatan: 800.0 kg
Nama            : Boeing 737
Berat           : 275.0 kg
Kapasitas       : 200 orang
Panjang Sayap   : 45.0 m
```

Tips BlueJ

Jika terminal BlueJ kalian sering hilang, tidak perlu restrart aplikasinya, cara berikut mungkin bisa jadi salah satu solusi.



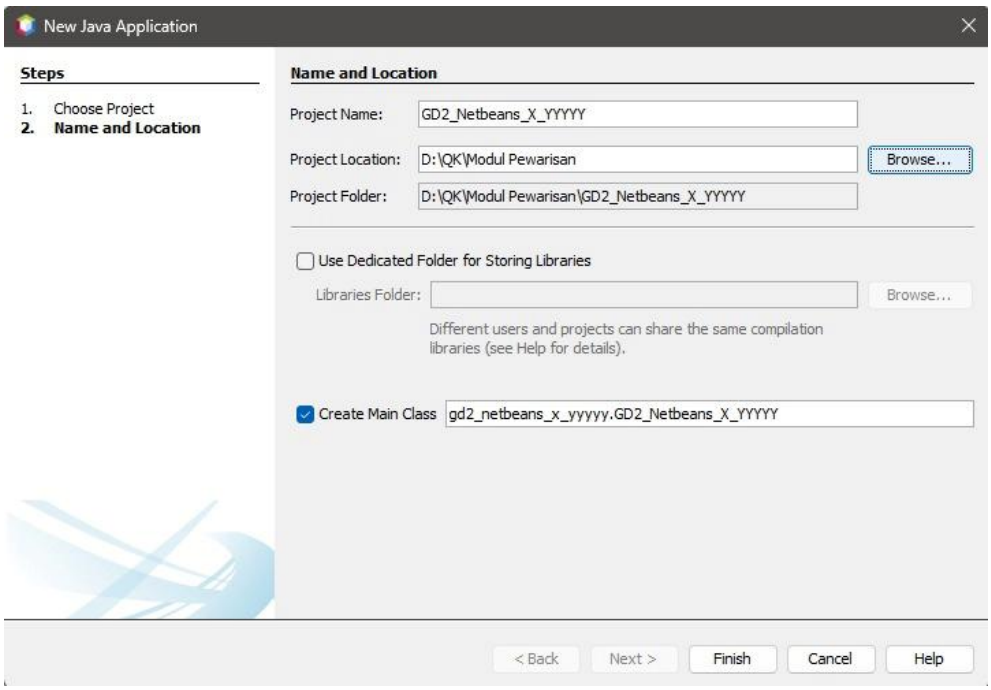
Silahkan buka task manager, bisa dengan shortcut Ctrl + shift + esc, kemudian cari OpenJDK, lalu ekspand. Disana kalian akan menemukan BlueJ: Terminal. Klik kanan dan pilih opsi maximize. Selamat, terminal kalian muncul lagi :)

GUIDED NETBEANS

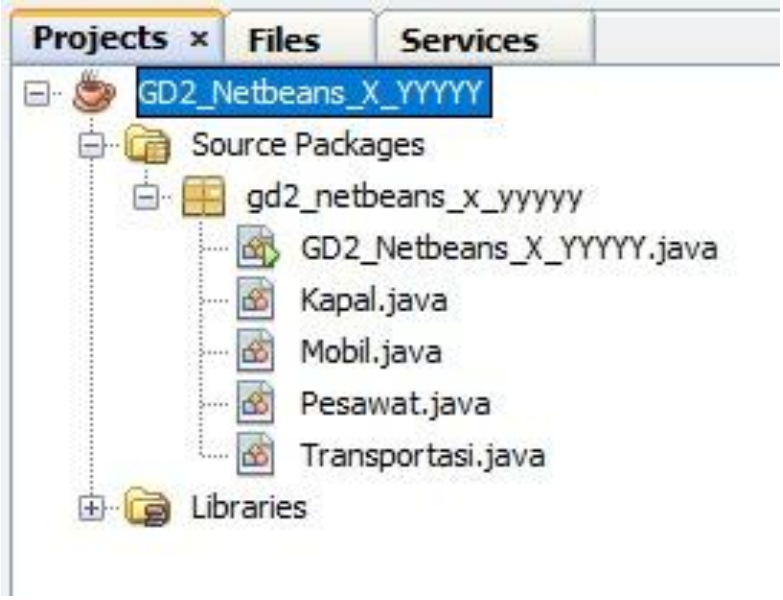
1. Buatlah project baru di netbeans dengan nama GD2_Netbeans_X_YYYYY.

   X = kelas

   Y = 5 digit NPM terakhir



2. Buatlah 4 file kelas baru dengan nama Transportasi, Mobil, Kapal, dan Pesawat. Struktur file akan seperti ini.



3. Buat kelas Transportasi terlebih dahulu sebagai kelas induknya dengan kode sebagai berikut.

```java
package gd2_netbeans_x_yyyyy;

/**
 *
 * @author nama lengkap praktikan
 */
public class Transportasi {
    protected String nama;
    protected double berat;
    protected int kapasitas;

    public Transportasi(String nama, double berat, int kapasitas) {
        this.nama = nama;
        this.berat = berat;
        this.kapasitas = kapasitas;
    }

    public void beroperasi()
    {
        System.out.println("Laporan [5 digit NPM praktikan], " + nama + " sedang beroperasi.");
    }

    public double hitungOperasional(double jarakKm)
    {
        return berat*jarakKm;
    }
}
```

4. Kemudian buat kelas anaknya. Pertama kita buat kelas Mobil.

```java
package gd2_netbeans_x_yyyyy;

/**
 *
 * @author nama lengkap praktikan
 */
public class Mobil extends Transportasi
{
    private int jumlahPintu;

    public Mobil(String nama, double berat, int kapasitas, int jumlahPintu)
    {
        super(nama, berat, kapasitas);
        this.jumlahPintu = jumlahPintu;
    }

    public void tampilDataMobil()
    {
        System.out.println("Nama            : " + nama);
        System.out.println("Berat           : " + berat + " kg");
        System.out.println("Kapasitas       : " + kapasitas + " orang");
        System.out.println("Jumlah Pintu    : " + jumlahPintu + " pintu");
    }

    public void cekMobil(double jarakKm)
    {
        double bobot = hitungOperasional(jarakKm) / 1000;
        System.out.println(this.nama + " memiliki bobot mobil: " + bobot);

        if(bobot < 5000)
        {
            System.out.println("Kategori: Mobil Ringan");
        }
        else
        {
            System.out.println("Kategori: Mobil Berat");
        }
    }
}
```

Perhatikan bagaimana kita menggunakan keyword extends untuk mewarisi kelas Transportasi. Perhatikan juga bagaimana kita menggunakan keyword super dalam memanggil konstruktor kelas induk. Kita juga dapat mengakses method kelas induk pada kelas anak karena visibilitasnya public.

5. Berikutnya, buat kelas Kapal yang mewarisi kelas Transportasi

```java
package gd2_netbeans_x_yyyyy;

/**
 *
 * @author nama lengkap praktikan
 */
public class Kapal extends Transportasi
{
    private double kapasitasMuatan;

    public Kapal(String nama, double berat, int kapasitas, double kapasitasMuatan)
    {
        super(nama, berat, kapasitas);
        this.kapasitasMuatan = kapasitasMuatan;
    }

    public void tampilDataKapal()
    {
        System.out.println("Nama            : " + nama);
        System.out.println("Berat           : " + berat + " kg");
        System.out.println("Kapasitas       : " + kapasitas + " orang");
        System.out.println("Kapasitas Muatan: " + kapasitasMuatan + " kg");
    }

    public void cekKapal(double jarakKm)
    {
        double operasional = hitungOperasional(jarakKm) + (kapasitasMuatan * 500);
        System.out.println(this.nama + " memiliki operasional kapal: " + operasional);

        if(operasional < 500000)
        {
            System.out.println("Kategori: Stabil");
        }
        else
        {
            System.out.println("Kategori: Berat");
        }

    }
}
```

6. Terakhir, buat juga kelas Pesawat yang mewarisi kelas Transportasi.

```java
package gd2_netbeans_x_yyyyy;

/**
 *
 * @author nama lengkap praktikan
 */
public class Pesawat extends Transportasi
{
    private double panjangSayap;

    public Pesawat(String nama, double berat, int kapasitas, double panjangSayap)
    {
        super(nama, berat, kapasitas);
        this.panjangSayap = panjangSayap;
    }

    public void tampilDataPesawat()
    {
        System.out.println("Nama            : " + nama);
        System.out.println("Berat           : " + berat + " kg");
        System.out.println("Kapasitas       : " + kapasitas + " orang");
        System.out.println("Panjang Sayap   : " + panjangSayap + " m");
    }

    public void cekPesawat(double jarakKm)
    {
        double operasional = hitungOperasional(jarakKm) + (kapasitas * 500) + (panjangSayap * 200);
        System.out.println(this.nama + " memiliki operasional pesawat: " + operasional);

        if(operasional <= 1500000)
        {
            System.out.println("Kategori: Standar");
        }
        else
        {
            System.out.println("Kategori: Tinggi");
        }

    }
}
```

7. Setelah selesai membuat semua kelas, sekarang mari kita implementasikan di main.

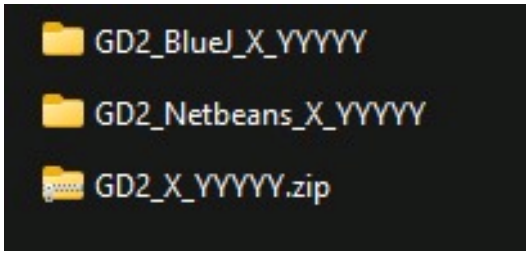
```java
package gd2_netbeans_x_yyyyy;

/**
 *
 * @author nama lengkap praktikan
 */
public class GD2_Netbeans_X_YYYYY {

    public static void main(String[] args)
    {
        //buat objek mobil, kapal, dan pesawat
        Mobil mobil = new Mobil("Suzuki Ignis", 100, 4, 4);
        Kapal kapal = new Kapal("Sabuk Nusantara", 350, 500, 800);
        Pesawat pesawat = new Pesawat("Boeing 737", 275, 200, 45);

        System.out.println("=== Method Parent [ beroperasi() ] ===");
        mobil.beroperasi();
        kapal.beroperasi();
        pesawat.beroperasi();
        System.out.println("");

        System.out.println("=== Hitung Operasional Transportasi ===");
        mobil.cekMobil(23);
        System.out.println("");
        kapal.cekKapal(45);
        System.out.println("");
        pesawat.cekPesawat(30);
        System.out.println("");

        System.out.println("=== Tampil Data Transportasi ===");
        mobil.tampilDataMobil();
        System.out.println("");
        kapal.tampilDataKapal();
        System.out.println("");
        pesawat.tampilDataPesawat();
    }
}
```

Jika output program yang tampil sama seperti di bawah ini, maka kode yang kalian tulis sudah benar.

```
Output - GD2_Netbeans_X_YYYYY (run)
run:
=== Method Parent [ beroperasi() ] ===
Laporan [5 digit NPM praktikan], Suzuki Ignis sedang beroperasi.
Laporan [5 digit NPM praktikan], Sabuk Nusantara sedang beroperasi.
Laporan [5 digit NPM praktikan], Boeing 737 sedang beroperasi.

=== Hitung Operasional Transportasi ===
Suzuki Ignis memiliki bobot mobil: 2.3
Kategori: Mobil Ringan

Sabuk Nusantara memiliki operasional kapal: 415750.0
Kategori: Stabil

Boeing 737 memiliki operasional pesawat: 117250.0
Kategori: Standar

=== Tampil Data Transportasi ===
Nama            : Suzuki Ignis
Berat           : 100.0 kg
Kapasitas       : 4 orang
Jumlah Pintu    : 4 pintu

Nama            : Sabuk Nusantara
Berat           : 350.0 kg
Kapasitas       : 500 orang
Kapasitas Muatan: 800.0 kg

Nama            : Boeing 737
Berat           : 275.0 kg
Kapasitas       : 200 orang
Panjang Sayap   : 45.0 m
BUILD SUCCESSFUL (total time: 0 seconds)
```

Pengumpulan

Kedua folder guided (blueJ & netbeans) yang sudah kalian buat disatukan kedalam sebuah folder dan di zip dengan format penamaan GD2_X_YYYYY.zip (jadi didalam folder utama ada 2 buah folder lagi yang bernama GD2_Netbeans_X_YYYYY dan GD2_BlueJ_X_YYYYY).



Kontak Asdos

Jika nantinya bingung dan ingin bertanya atau berdiskusi, silahkan kontak ke

- Teams 230712372@students.uajy.ac.id (Adela Desca)
- Whatsapp 082199193919 (disarankan)